

Respostas do Aquecimento para a Prova 1 (P1) – Grafos (INE5413)

Ciências da Computação – Universidade Federal de Santa Catarina

Prof. Rafael de Santiago

Nome:

Matrícula:

Observações gerais:

- A prova deverá ser concluída até as 10h00m;
- Pode ser utilizado material para consulta;
- Não será permitido compartilhamento de material de consulta.

1. (2.5pt) Considere o seguinte problema: Em um setor de investigações da polícia em uma determinada cidade, têm-se acesso às seguintes informações de ligações telefônicas que acontecem na área: dados sobre os contatos telefônicos (nome e CPF do proprietário da linha); listagem de quem ligou para quem dentro da mesma cidade; e duração de cada ligação. Recentemente, há uma demanda no setor relacionada a identificar as pessoas próximas a um indivíduo sendo investigado. No contexto de ligação telefônica, os investigadores consideram apenas ligações com mais de 30 segundos. São consideradas pessoas próximas, a distância de até duas ligações. Ou seja, se o indivíduo A ligou para B que ligou para C e as ligações duraram mais de 30 segundos cada, considera-se que A, B e C são indivíduos “próximos” para o contexto investigativo, inclusive se A e C não tiverem registros de ligações com mais de 30 segundos entre si. Com base nesse problema, responda:

- (a) (0.5pt) Qual tipo de grafo poderia ser utilizado para o problema (dirigido ou não, ponderado ou não)? Justifique.

Resposta: Para representar o problema, se utilizaria um grafo não-dirigido e não-ponderado. Não-ponderado porque é importante tomar nota se houve ou não ligações entre dois indivíduos. Não-dirigido porque a origem e o destino da ligação é ignorada, já que se considera que os dois envolvidos são considerados próximos apenas por entrarem em contato.

- (b) (1.0pt) Como seria a montagem do grafo de entrada para o problema?

Resposta: Cria-se um vértice $G = (V, E)$, onde V é o conjunto de indivíduos e E é o conjunto de ligações, no qual cada aresta $\{u, v\} \in E$ representa uma ligação telefônica que durou mais de 30s entre os indivíduos u e v .

- (c) (1.0pt) Qual dos algoritmos estudados poderia ser utilizado? O que deveria ser modificado nesse algoritmo?

Resposta: Devido a propriedade de caminho mínimo para grafos não-ponderados (Teorema 3.1.5 nas Anotações da Disciplina), encontra-se a conexão entre cada par de indivíduos com o menor número de arestas através de uma busca em largura. Então, utilizaria uma busca em largura modificada. Essa busca não incluiria vizinhos de vértices com $d_u \geq 2$ na fila.

2. (2.5pt) Considere o seguinte problema: Uma empresa quer participar de um edital para conectar várias localidades utilizando uma linha férrea. No levantamento, há 1000 localidades denominadas de $l_1, l_2, \dots, l_{1000}$, nas quais l_1 é o começo e o destino da linha férrea. Uma das questões principais para participar da licitação é ser competitivo no preço da grande obra. Nesse contexto, a empresa deseja encontrar um ciclo que atenda as localidades sem a necessidade de construir pontes, atravessar morros e realizar grandes desapropriações de terra. Para isso, os técnicos da empresa se reuniram e definiram uma listagem de pares de localidades que não poderiam ser conectadas pois inviabilizariam o projeto. Deseja-se então desenvolver um programa de computador que faça o cálculo se é viável ainda conectar todos os pontos através de um ciclo. Com base no relatado acima, responda:

- (a) (1.25pt) Como seria a montagem do grafo de entrada para o problema?

Resposta: Um grafo $G = (V, E)$ não-dirigido e não-ponderado seria criado, no qual V seria o conjunto de localidades. A criação do conjunto de arestas se daria em dois passos: (i) primeiro, consideraria-se que cada par de vértices são conectados (grafo completo), então $E = V \times V$; (ii) depois, removeria-se as arestas correspondentes à listagem de pares de localidades que não poderia ser conectadas.

aresta entre todos os pares de vértices possíveis

- (b) (1.25pt) Qual dos algoritmos estudados poderia ser utilizado? O que deveria ser modificado nesse algoritmo?

Resposta: Como se necessita de um ciclo que passe por todas as localidades e dadas as restrições de custo, considera-se encontrar um ciclo hamiltoniano. Portanto, utilizaria-se o algoritmo de Bellman-Held-Karp para procurá-lo.

o foco são as localidades (vértices); as arestas, neste caso, a empresa não quer passar por todos os possíveis

3. (2.5pt) Sabe-se que o algoritmo de Floyd-Warshall tem complexidade de tempo $\Theta(|V|^3)$ para encontrar os caminhos de menor custo entre todos os vértices de um grafo $G = (V, E, w : E \rightarrow \mathbb{R})$. Considerando que G tenha $|E| \approx |V|$ e não tenha arestas com pesos negativos, faça:

$n \cdot E \approx n \cdot V \rightarrow$ não tem muitas conexões $\rightarrow$ pode indicar dijkstra

- (a) (1.5pt) Crie um algoritmo mais eficiente que Floyd-Warshall para encontrar os caminhos mínimos para cada par de vértices em G . Se for necessário, considere que está a disposição os algoritmos de Bellman-Ford (complexidade de tempo $O(|V||E|)$) e Dijkstra (complexidade de tempo $O(|V| + |E|) \log_2 |V|$)

Resposta: O algoritmo para atender a solicitação se encontra abaixo. Nele, descobre-se o custo dos menores caminhos a partir de cada vértice para cada outro em G .

cam. mín. de ponto único $\rightarrow$ fazer p1 todos os pontos

Algorithm 1: Algoritmo da Questão 3(a).

Input : um grafo ponderado $G = (V, A, w)$

```

1 criar matriz de distâncias  $D$  com  $\mathbb{R}^{|V| \times |V|}$   $\rightarrow$  (comum em todos p1 todos) } inicializar
2 preencher matriz com  $d_{uv} \leftarrow \infty$  para todo  $u, v \in V$ 
3 foreach  $v \in V$  do  $\rightarrow$  p1 cada vértice
4    $(D', A') \leftarrow \text{Dijkstra}(G, v)$   $\rightarrow$  lista de distâncias e ancestrais de  $v$  p1 todos
5   foreach  $u \in V$  do
6      $d_{vu} \leftarrow D'_u$   $\rightarrow$  distâncias atualizadas na matriz
7 return  $D$ 

```

- (b) (0.5pt) Calcule a complexidade de tempo do algoritmo produzido;

Resposta: A definição inicial da matriz nas linhas 1 e 2 consome $\Theta(|V|^2)$ operações. O laço da linha 3 realiza $|V|$ vezes a chamada do algoritmo de Dijkstra ($O(|V| + |E|) \log_2 |V|$) e

$|V|$ vezes um laço de repetição que passa sobre cada vértice. Como $|E| \approx |V|$, cada chamada do Dijkstra demandaria $O(|V| \log_2 |V|)$. Então, têm-se a complexidade de $O(|V|^2 \log_2 |V|)$ que é dominada assintoticamente por $\Theta(|V|^3)$, ou seja, o algoritmo proposto é mais eficiente que Floyd-Warshall.

- (c) (0.5pt) Caso não tenha usado um dos algoritmos dispostos (Bellman-Ford ou Dijkstra), justifique.

Resposta: O uso de Bellman-Ford se justificaria apenas se o grafo de entrada possuísse arestas de peso negativo, pois Dijkstra é mais eficiente que Bellman-Ford para grafos com pesos não-negativos.

4. (2.5pt) Seja $G = (V, A, w)$ um grafo dirigido e ponderado, crie um algoritmo para determinar os caminhos mínimos a partir de todo $v \in V$ para um vértice t . Caso seja necessário, pode-se chamar os algoritmos Bellman-Ford (complexidade de tempo $O(|V||E|)$) e Dijkstra (complexidade de tempo $O(|V| + |E|) \log_2 |V|$) como se já estivessem implementados.

Resposta: O algoritmo para atender a solicitação se encontra abaixo. No algoritmo, por inverter o sentido dos arcos, ao executar um algoritmo como Bellman-Ford ou Dijkstra com t como vértice de "origem" (agora destino), se obtém os caminhos mínimos de todos os vértices para t .

Algorithm 2: Algoritmo da Questão 4.

```

Input : um grafo dirigido e ponderado  $G = (V, A, w)$ , um vértice de destino  $t$ 
1  $A' \leftarrow \{\}$   $\rightarrow$  novo conjunto de arestas
2 criar função  $w' : A' \rightarrow \mathbb{R}$   $\rightarrow$  e os pesos
3 foreach  $(u, v) \in A$  do  $\rightarrow$  P! toda aresta
4    $A' \leftarrow A' \cup \{(v, u)\}$   $\rightarrow$  Criado conexão contrário
5   define  $w'((v, u)) \rightarrow w((u, v))$   $\rightarrow$  de mesmo peso
6  $G' \leftarrow (V, A', w')$   $\rightarrow$  grafo c/ vértices originais, mas com arestas invertidas
   /*  $D_u$  define as distâncias a partir de  $u$  até  $t$  e  $S_u$  seria o sucessor de  $u$  no caminho
      até  $t$ . */
7  $(D, S) \leftarrow \text{Bellman-Ford}(G', t)$   $\rightarrow$  escolhido qz ã diz
8 return  $(D, S)$   $\rightarrow$  Se tem negativo

```

$\hookrightarrow$ engana BF ao mandar " t " como origem, quando na verdade é um destino; Fonte única vira destino único

Boa Prova!